

Стать IT-шником может каждый



Предварительная обработка и очистка данных в Pandas



Преподаватель



Ваше фото

ИМЯ ФАМИЛИЯ

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



ВАЖНО!

- Камера должна быть включена на протяжении всего занятия.
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях.
- Вести себя уважительно и этично по отношению к остальным участникам занятия.
- Во время занятия будут интерактивные задания, будьте готовы включить микрофон или демонстрацию экрана по просьбе преподавателя.

ПЛАН ЗАНЯТИЯ

Введение

Основной блок

Задание для закрепления

Вопросы по основному блоку

Заключение



Стать IT-шником может каждый



ПОВТОРЕНИЕ ИЗУЧЕННОГО





Повторение

- Box plot
- Пример анализа данных. Box plot
- Violin plot
- Heatmap
- Clustermap



Стать IT-шником может каждый



ВОПРОСЫ





Введение

Изучение DataFrame и определение отсутствующих значений

Обработка пропущенных значений

Подстановка недостающих значений

Удаление дубликатов

Переименование столбцов

Изменение типов данных

Нормализация текстовых данных



Стать IT-шником может каждый



ОСНОВНОЙ БЛОК



Изучение DataFrame и определение отсутствующих значений



Загрузка датасета [data_for_cleaning](#)

```
import numpy as np
import pandas as pd

df = pd.read_csv('.\data_for_cleaning.csv')
print(df.head(5))
print(df.tail(5))
print(df.info())
print(df.describe())
```

Стать IT-шником может каждый



Определение отсутствующих значений

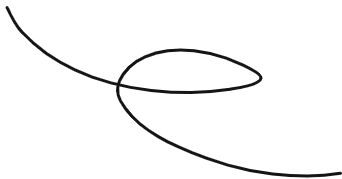
Проблема качества данных: пропущенные значения

В pandas для обнаружения отсутствующих значений в DataFrame используются методы `isnull()` и `notnull()`



isnull() —

это метод, который возвращает булевый DataFrame той же формы, что и исходный DataFrame, в котором каждая ячейка имеет значение True, если соответствующая ячейка в исходном DataFrame имеет значение NaN (Not a Number) или None, и False в противном случае.



Метод isnull()

- Синтаксис: `DataFrame.isnull()`
- Возвращает: `DataFrame` с булевыми значениями (`True` для пропущенных значений и `False` для непропущенных)
- Параметры: Этот метод не принимает никаких параметров

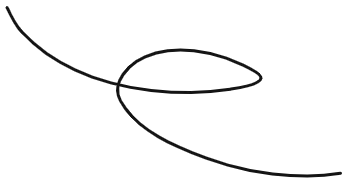
Пример метода isnull()

```
missing_values = df.isnull()  
print(missing_values)
```



notnull() —

это метод, обратный методу `isnull()`. Возвращает булевый `DataFrame` той же формы, что и исходный `DataFrame`, где каждая ячейка имеет значение `True`, если соответствующая ячейка в исходном `DataFrame` не `NaN` (Not a Number) или `None`, и `False` в противном случае.



Метод notnull()



- Синтаксис: `DataFrame.notnull()`
- Возвращает: `DataFrame` с булевыми значениями (`True` для не пропущенных значений и `False` для пропущенных значений)
- Параметры: Этот метод не принимает никаких параметров

Пример метода notnull()

```
non_missing_values = df.notnull()  
print(non_missing_values)
```

Использование датасета

```
missing_values = df.isnull()

non_missing_values = df.notnull()

print("Пропущенные значения:")
print(missing_values)

print("\nНепропущенные значения:")
print(non_missing_values)
```


Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Обработка пропущенных значений



Стать IT-шником может каждый

Обработка пропущенных значений



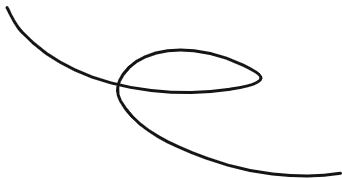
Часть процесса очистки данных: обработка пропущенных значений

В pandas для обработки пропущенных значений используются методы `fillna()` и `dropna()`



fillna() —

это метод, который используется для заполнения отсутствующих значений в DataFrame указанными значениями. Позволяет заменить NaN (Not a Number) или отсутствующие значения на указанное скалярное значение, словарь или использовать различные методы, такие как 'ffill' (прямое заполнение) или 'bfill' (обратное заполнение).



Метод fillna()



- Синтаксис: `DataFrame.fillna(value=None, method=None, axis=None, inplace=False, limit=None, downcast=None)`
- Возвращает: `DataFrame` с заполненными или замененными недостающими значениями

Параметры метода fillna()

Параметр	Пояснение
value	Скаляр, dict, Series или DataFrame. Значение, используемое для заполнения недостающих значений
метод	{'backfill', 'bfill', 'pad', 'ffill', None}, по умолчанию None. Метод, используемый для заполнения пропущенных значений
axis	{0 или 'index', 1 или 'columns'}. Ось, вдоль которой заполняются пропущенные значения
inplace	bool, по умолчанию False. Модифицировать DataFrame на месте (не создавать новый объект)
limit	int, по умолчанию None. Максимальное количество последовательных NaN для заполнения
downcast	dict, по умолчанию None. Словарь item→dtype с тем, что нужно передать вниз, если это возможно

Пример метода fillna()

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
```

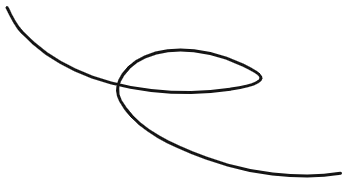
```
df['Gender'].fillna('Unknown', inplace=True)
```

```
df['Department'].fillna(method='ffill', inplace=True)
```



dropna() —

это метод, который используется для удаления отсутствующих значений из DataFrame. Позволяет удалять строки или столбцы, в которых любой или все элементы являются NaN (Not a Number).



Метод dropna()



- Синтаксис: `DataFrame.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False)`
- Возвращает: `DataFrame` с удаленными недостающими значениями

Параметры метода `dropna()`

Параметр	Пояснение
<code>axis</code>	{0 или 'index', 1 или 'columns'}. Ось, по которой нужно отбросить пропущенные значения
<code>how</code>	{'any', 'all'}, по умолчанию 'any'. Определяет, удалять ли строку или столбец из DataFrame, где любой или все элементы являются NaN
<code>thresh</code>	int, по умолчанию None. Требуется, чтобы количество значений, не являющихся NaN, было максимальным
<code>subset</code>	array-like, по умолчанию None. Метки по другим осям, которые необходимо учитывать, например, если вы удаляете строки, то это будет список столбцов, которые необходимо включить
<code>inplace</code>	bool, по умолчанию False. Модифицировать DataFrame на месте (не создавать новый объект)

Пример метода dropna()

```
# Удаляем строки, в которых отсутствует какой-либо элемент в DataFrame
```

```
df.dropna(inplace=True)
```

```
# Удаление строк, в которых все элементы отсутствуют в DataFrame
```

```
df.dropna(how='all', inplace=True)
```

```
# Отбрасываем строки, в которых столбец 'Зарплата' или 'Пол' имеет недостающие значения
```

```
df.dropna(subset=['Salary', 'Gender'], inplace=True)
```

Использование датасета

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Gender'].fillna('Unknown', inplace=True)
df['Department'].fillna(method='ffill', inplace=True)
df.dropna(inplace=True)
print(df)
```


Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Подстановка недостающих значений



Стать IT-шником может каждый



Подстановка недостающих значений

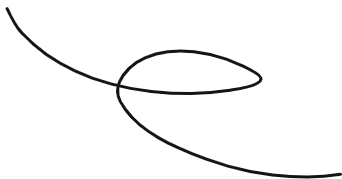
Подход к обработке отсутствующих данных: подстановка недостающих значений

Pandas предоставляет различные методы для подстановки отсутствующих значений с использованием таких статистических показателей, как среднее, медиана и мода



Использование среднего значения —

это метод, который используется для подстановки пропущенных значений. Можно использовать среднее значение для замены пропущенных значений в числовых столбцах.



Использование среднего значения



- Синтаксис: `DataFrame['имя_столбца'].fillna(value=df['имя_столбца'].mean(), inplace=True)`
- Возвращает: DataFrame с отсутствующими значениями в указанном столбце, замененными средним значением

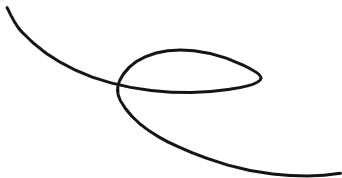
Пример использования среднего значения

```
df['Age'].fillna(value=df['Age'].mean(), inplace=True)
```



Медиана —

это мера, которая представляет собой срединное значение в наборе данных. Можно использовать медиану для замены пропущенных значений в числовых столбцах, особенно если данные содержат провалы.



Использование медианы



- Синтаксис: `DataFrame['имя_столбца'].fillna(value=df['имя_столбца'].median(), inplace=True)`
- Возвращает: DataFrame с отсутствующими значениями в указанном столбце, замененными медианным значением

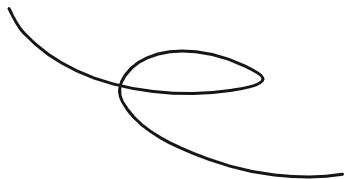
Пример использования медианы

```
df['Salary'].fillna(value=df['Salary'].median(), inplace=True)
```



Мода —

это мера, которая представляет собой наиболее часто встречающееся значение в наборе данных. Можно использовать для замены отсутствующих значений в категориальных столбцах.



Использование моды



- Синтаксис: `DataFrame['имя_столбца'].fillna(value=df['имя_столбца'].mode()[0], inplace=True)`
- Возвращает: DataFrame с отсутствующими значениями в указанном столбце, замененными значением режима

Пример использования моды

```
df['Gender'].fillna(value=df['Gender'].mode()[0], inplace=True)
```

Использование датасета

```
df['Age'].fillna(value=df['Age'].mean(), inplace=True)
df['Salary'].fillna(value=df['Salary'].median(), inplace=True)
df['Gender'].fillna(value=df['Gender'].mode()[0], inplace=True)
print(df)
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Удаление дубликатов



Стать IT-шником может каждый



ICH
S*



Удаление дубликатов

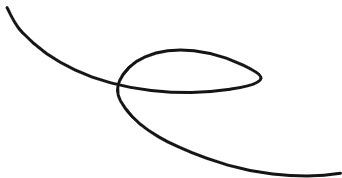
Проблема искажения результатов анализа: дублирующиеся строки

Pandas предоставляет метод `drop_duplicates()`, чтобы легко удалить дубликаты строк из DataFrame



drop_duplicates() —

это метод, который используется для удаления дубликатов строк из DataFrame на основе указанных столбцов или всех столбцов.



Метод drop_duplicates()



- Синтаксис: `DataFrame.drop_duplicates(subset=None, keep='first', inplace=False, ignore_index=False)`
- Возвращает: `DataFrame` с удаленными дублирующимися строками

Параметры метода `drop_duplicates()`

Параметр	Пояснение
<code>subset</code>	Метка столбца или последовательность меток, необязательно. Подмножество столбцов для выявления дубликатов. По умолчанию учитываются все столбцы
<code>keep</code>	{'first', 'last', False}, по умолчанию 'first'. Определяет, какой дубликат сохранять. 'first' сохраняет первое вхождение, 'last' сохраняет последнее вхождение, а False удаляет все дубликаты
<code>inplace</code>	bool, по умолчанию False. Модифицировать DataFrame на месте (не создавать новый объект)
<code>ignore_index</code>	bool, по умолчанию False. Если значение True, то результирующий DataFrame будет иметь новый RangeIndex

Пример использования метода drop_duplicates()

```
df.drop_duplicates(inplace=True)
```

```
df.drop_duplicates(subset=['Name', 'Age'], inplace=True)
```

Использование датасета

```
print("Изначальный DataFrame:")  
print(df)  
  
df.drop_duplicates(inplace=True)  
  
print("\nDataFrame после удаления дубликатов:")  
print(df)
```


Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



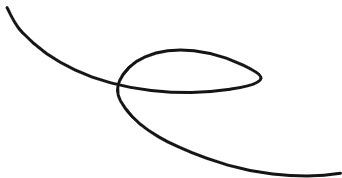
Переименование столбцов





Переименование столбцов —

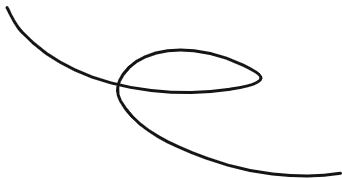
это распространенная операция при работе с pandas DataFrames, для того, чтобы сделать имена столбцов более описательными или исправить опечатки или несоответствия в именах столбцов.





rename() —

это метод, который позволяет переименовывать столбцы в DataFrame, указывая словарь, который сопоставляет текущие имена столбцов с новыми именами столбцов.



Метод rename()



- Синтаксис: `DataFrame.rename(columns={'текущее_имя':'новое_имя'}, inplace=False)`
- Возвращает: `DataFrame` с переименованными столбцами

Параметры метода rename()

Параметр	Пояснение
columns	Диктоподобный объект или функция, необязательно. Словарь или функция для переименования меток осей
inplace	bool, по умолчанию False. Модифицировать DataFrame на месте (не создавать новый объект)

Использование датасета

```
print("Изначальный DataFrame:")  
print(df)  
  
df.rename(columns={'Joining_Date': 'JoinDate'}, inplace=True)  
  
print("\nDataFrame после переименования поля:")  
print(df)
```


Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



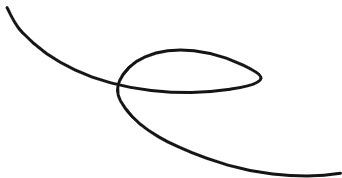
Изменение типов данных





Изменение типов данных —

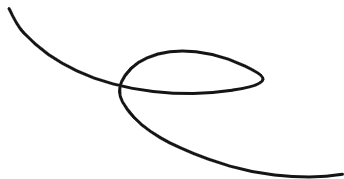
это распространенная операция при работе с pandas DataFrames, особенно для преобразования столбцов в соответствующие типы данных для целей анализа или визуализации.





astype() —

это метод, который используется для изменения типов данных столбцов в DataFrame на указанный тип данных.



Метод astype()

- Синтаксис: `DataFrame.astype(dtype, copy=True, errors='raise')`
- Возвращает: `DataFrame` с измененными типами данных



Параметры метода `astype()`

Параметр	Пояснение
<code>dtype</code>	Тип данных, или dict из имени столбца и типа данных. Тип данных, в который нужно преобразовать DataFrame. Это может быть один тип данных для всех столбцов или словарь, определяющий тип данных для каждого столбца
<code>copy</code>	bool, по умолчанию True. Возвращать ли новый DataFrame с указанными типами данных или модифицировать существующий DataFrame на месте
<code>errors</code>	{'raise', 'ignore'}, по умолчанию 'raise'. Определяет, как обрабатывать ошибки при преобразовании типов данных. Если 'raise', то при любой ошибке преобразования типа данных будет выдана ошибка. Если 'ignore', то ошибки будут проигнорированы, и будут возвращены исходные данные

Использование датасета

```
print("Изначальный DataFrame:")
print(df)
print("\nТипы данных:")
print(df.dtypes)

df['Age'] = df['Age'].astype(float)

df['Rating'] = df['Rating'].astype(int)

print("\nDataFrame после изменения типов данных:")
print(df)
print("\nТипы данных после изменения:")
print(df.dtypes)
```


Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



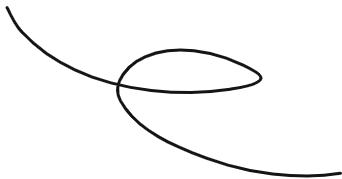
Нормализация текстовых данных





Нормализация текстовых данных —

это этап предварительной обработки, который часто выполняется перед задачами обработки естественного языка (NLP): классификация текстов, анализ настроения или моделирование тем. Включает в себя преобразование текстовых данных к единообразному формату путем удаления строчных букв и специальных символов.

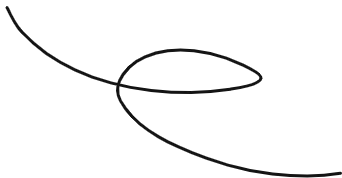




Перевод текстовых данных в нижний регистр —

это метод, переводящий в нижний регистр все символы в тексте.

Обеспечивает согласованность текстовых данных и помогает снизить сложность задач обработки текста.



Перевод текстовых данных в нижний регистр



- Синтаксис: `DataFrame['текстовое_поле'].str.lower()`
- Возвращает: Серия с текстовыми данными, преобразованными в нижний регистр

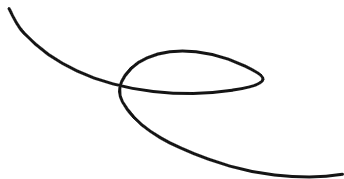
Пример перевода текстовых данных в нижний регистр

```
df['TextColumn'] = df['TextColumn'].str.lower()
```

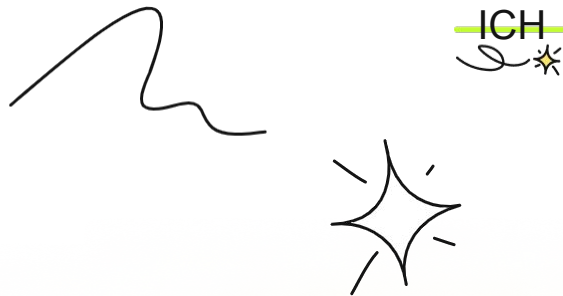



Удаление специальных символов —

это метод, включающий в себя удаление неалфавитно-цифровых символов, знаков препинания и других символов, которые могут быть не важны для задач анализа текста.



Удаление специальных символов



- Синтаксис: `DataFrame['текстовое_поле'].str.replace(r'^\w\s', '')`
- Возвращает: Серия, в которой из текстовых данных удалены специальные символы
- Параметр: `r'^\w\s'`: Шаблон регулярного выражения, который соответствует любому символу, не являющемуся символом слова или пробелом

Пример удаления специальных символов

```
df['TextColumn'] = df['TextColumn'].str.replace(r'^\w\s', "")
```

Использование датасета

```
import pandas as pd

df = pd.DataFrame({
    'TextColumn': ['Это образец ТЕКСТА со СПЕЦИАЛЬНЫМИ символами!',
                  'Еще один образец, с пунктуацией!!!!',
                  'Нормализация текстовых данных важна для задач НЛП']
})

df['TextColumn'] = df['TextColumn'].str.lower()

df['TextColumn'] = df['TextColumn'].str.replace(r'[^\w\s]', '', regex=True)

print(df)
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ПРАКТИЧЕСКАЯ РАБОТА



Практическое задание

Что следует изменить в следующем датасете? Как это можно сделать?

	Name	Age	Gender	Height (cm)	Weight (kg)
0	Alice	25.0	Female	160.0	55.0
1	Bob	30.0	Male	175.0	NaN
2	Charlie	NaN	Male	180.0	75.0
3	David	35.0	Male	NaN	80.0
4	Emily	40.0	NaN	165.0	60.0

Решение

```
import pandas as pd
import numpy as np

df['Age'] = df['Age'].astype('Int64')
df['Gender'] = df['Gender'].astype('category')

df['Height (cm)'].fillna(df['Height (cm)'].mean(), inplace=True)
df['Weight (kg)'].fillna(df['Weight (kg)'].median(), inplace=True)

df.rename(columns={'Height (cm)': 'Height', 'Weight (kg)': 'Weight'}, inplace=True)

print("\nCleaned DataFrame:")
print(df)
```


Стать IT-шником может каждый



ОСТАВШИЕСЯ ВОПРОСЫ



Полезные ссылки

- [A Beginner level methods to clean messy data](#)
- [Pythonic Data Cleaning With pandas and NumPy](#)

Стать IT-шником может каждый



ЗАКЛЮЧЕНИЕ

